

Post Quantum Cryptography

Spring 2025

Table of Contents

1. Lecture 1	5
1.1. Introduction	5
1.1.1. Note: Some general information	5
1.1.2. Note: Grading	5
1.1.3. Note	5
1.1.4. Concept: Short Integer Solutions	6
1.1.5. Definition: B short solutions	6
1.2. Short Integer Solutions	6
1.2.1. Definition: $\text{SIS}_{n,m,B,q}$	6
1.2.2. <i>Proposition</i> : existence of B short solutions	7
1.2.3. Definition: Collision resistant hash functions (CRHF)	7
1.2.4. <i>Theorem</i> : Collision Resistant hash function using SIS assumption	7
1.2.5. Definition: inhomogeneous SIS, (ISIS)	8
1.2.6. Note	8
1.2.7. Definition: Trapdoor	8
1.2.8. Note: Why over $\mathbb{R}$ specifically	8
2. Lecture 2 Digital Signatures from I-SIS	9
2.0.1. Note: Recap: SIS and I-SIS	9
2.0.2. Note: Trapdoors and TrapGen	9
2.0.3. Definition: statistical distance (denoted approx_s) (??)	9
2.0.4. Note: A nice property 1 about TrapGen	9
2.0.5. Note	9
2.0.6. Note: A nice property 2 about TrapGen	9
2.0.7. Note: A nice property 3 about TrapGen	9
2.0.8. Definition: Digital Signatures	10
2.0.9. Concept: Digital Signature scheme from I-SIS	10
2.0.10. Definition: Formal Description of digital signature scheme (??)	10
3. Lecture 3: LWE and LWR	11
3.0.1. Note: Last time	11
3.0.2. Definition: $\text{SearchLWE}(n,m,q,\chi)$	11
3.0.3. Note	11
3.0.4. Definition: $\text{DecisionLWE}(n,m,q,\chi)$	11
3.0.5. Definition: public key encryption	11
3.0.6. Definition: Oded Regev public key encryption scheme	12
3.0.7. <i>Proposition</i> : Security of Definition 3.0.6	13
3.0.8. Definition: dual regev public key encryption scheme	13
3.1. LWR	13
3.1.1. Note: Intuition	13
3.1.2. Definition: rounding function $\lfloor \cdot \rfloor : \mathbb{Z}_p \rightarrow \mathbb{Z}_q$	14
4. Lecture 4: LWR, PRF, reductions	15

4.0.1.	Definition: LWR	15
4.0.2.	Note: Fact 1	15
4.0.3.	Note: Fact 2	15
4.0.4.	Definition: pseudorandom functions	15
4.0.5.	Note	15
4.0.6.	Notation	15
4.0.7.	Definition: Construction 1 of PRF using LWR	16
4.0.8.	<i>Theorem</i>	16
4.0.9.	Example: Recursive computation of 4.0.7 PRF	16
4.0.10.	Definition: Construction 2 of PRF using LWR	17
4.1.	Reductions between SIS, LWE, LWR	17
4.1.1.	Definition: Easy to Solve	17
4.1.2.	Definition: SIS (search) is easy	17
4.1.3.	Definition: Decision LWE is easy	17
4.1.4.	Definition: Search LWE is easy	17
4.1.5.	<i>Theorem</i>	18
4.1.6.	<i>Theorem</i>	18
5.	Lecture 5: more reductions, lattices	19
5.0.1.	<i>Theorem</i>	19
5.0.2.	<i>Theorem:</i> Whether s in LWE comes from uniform or chi (small) distribution doesn't matter	20
5.0.3.	<i>Theorem:</i> LWR is easy implies LWE is easy	20
5.1.	Lattices	20
5.1.1.	Note	20
5.1.2.	Definition: Lattices	20
5.1.3.	Definition: Lattice basis characterization	21
5.1.4.	Definition: Fundamental Parallelepiped	21
5.1.5.	Definition: shortest vector in lattice	21
5.1.6.	Definition: $\lambda_i(\mathcal{L})$	21
5.1.7.	Note	21
5.1.8.	Definition: Coset of a lattice	21
5.1.9.	Definition: Dual Lattice	21
5.1.10.	Definition: Shorted vector problem	21
5.1.11.	Definition: GapSVP $_{\gamma}$ / ApproximateSVP $_{\gamma}$	22
5.1.12.	<i>Theorem:</i> Hardness of GapSVP $_{\gamma} \Rightarrow$ hardness of SIS $_{n,m,q,B}$ (intuition)	22
5.1.13.	Definition: Discrete Gaussian function over $\mathbb{R}^n$	22
5.1.14.	Definition: Discrete Gaussian distribution	23
5.1.15.	Problem 0	23
5.1.16.	<i>Theorem</i>	23
6.	Lecture 6	24
6.0.1.	Note	24
7.	Lecture 7 (online)	25
7.0.1.	Note	25
8.	Lecture 8 (youtube)	26
8.0.1.	Note	26
9.	Lecture 9 - Insecure Ring-SIS	27
9.0.1.	Note: Recap	27

9.0.2.	Definition: Structured matrices $X, X^2, \dots$ generate a lattice which is isomorphic to univariate polynomial quotient rings	28
9.0.3.	Notation	28
9.0.4.	Example	28
9.0.5.	Definition: New Hash Function	29
9.0.6.	Definition: Ring SIS (insecure version)	29
9.0.7.	<i>Theorem</i>	30
9.0.8.	<i>Lemma</i> : high chance of factorability of	30
10.	Lecture 10: ring LWE priv/pub encryption, code based assumptions and error correcting codes	31
10.0.1.	Definition: Rings SIS (“regular”)	31
10.0.2.	Note	31
10.0.3.	Definition: lattice generate by $\mathbb{Z}_q[x]/x^n + 1$	31
10.0.4.	Definition: Ring LWE	31
10.0.5.	Definition: Private Key Enc using ring LWE	32
10.0.6.	Definition: Public Key Enc using ring LWE	32
10.0.7.	Definition: prisoner’s game	32
10.1.	error correcting codes	32
11.	Lecture 11: Learning Parity with noise/error correcting codes, Peikert	33
12.	Lecture 12: LPN based hash functions, cryptographic group actions	34
12.0.1.	Definition: Hash function (LPN)	34
12.0.2.	<i>Proposition</i> : Security of 12.0.1	35
12.1.	cryptographic group actions	36
12.1.1.	Definition: cryptographic group action	36
12.1.2.	Example: group action	36
12.1.3.	Note	36
12.1.4.	Definition: regular group actions	36
12.1.5.	Definition: ‘effective group actions’ (EGA)	36
12.1.6.	Definition: One-way EGA	36
12.1.7.	Definition: Weak Unpredictable EGA	37
12.1.8.	Definition: Weak Pseudorandom EGA	37
12.1.9.	Definition: Pseudorandom functions from EGA	37
12.1.10.	<i>Proposition</i> : proof of security of 12.1.9 in the simple case	37
12.1.11.	Definition: Public Key Encryption from EGA	38
12.1.12.	<i>Proposition</i>	38
13.	Lecture 13: Quantum Query Security	39
13.0.1.	Notation: Oracle to access to function f	39
13.0.2.	Definition: Pseudorandom Function	39
13.0.3.	Definition: Random Oracle Model	39
13.0.4.	Note	39
13.0.5.	Definition: Quantum Oracle Access (standard oracle)	40
13.0.6.	Definition: Phase Oracle	40
13.0.7.	<i>Proposition</i> : Extracting s from $\langle x, s \rangle$ (Bernstein-Vazirani)	41
13.0.8.	Note	41
13.1.	Quantum Query Secure Pseudorandom functions	42
13.1.1.	Notation: quantum query access	42
13.1.2.	Concept: Quantum Query Security	42
13.1.3.	Definition: GGM construction of PRF	43

13.1.4.	Definition: Indistinguishability of distributions	43
13.1.5.	Definition: Oracle Indistinguishability	43
13.1.6.	<i>Theorem</i> : distinguishable iff oracle-distinguishable	44
13.1.7.	Definition: Small Range Distribution	44
13.1.8.	<i>Lemma</i> : Small Range	44
13.1.9.	Note	44
13.1.10.	<i>Theorem</i>	45
14.	Lecture 14 Quantum Query Security continued, signatures	46
14.0.1.	Definition: Digital signature security (classical)	46
14.0.2.	Definition: Quantum Digital Signature Security	46
14.0.3.	<i>Proposition</i> : Equivalent methods for implementing a randomized sign	46
14.0.4.	Concept: Separation	47
14.0.5.	Definition: Separation Signature Scheme	47
14.0.6.	<i>Theorem</i> : 14.0.5 is not secure according to 14.0.2	48
14.0.7.	Note	48
14.1.	Signature scheme satisfying quantum query security	48
14.1.1.	Definition: Chameleon Hash Function	49
14.1.2.	Note	49
14.1.3.	Note	49
14.1.4.	Definition: sig scheme (quantum query secure)	49
14.1.5.	<i>Theorem</i> : security of 14.1.4	50

1. Lecture 1

1.1. Introduction

1.1.1. Note: Some general information

Google: willow quantum chip (superconducting qubits)

Microsoft: Majorana 1 quantum chip (topological qubits)

Jensen Huang: changed his mind on feasibility of quantum computers from infeasible to possibly feasible

Shor's (efficient) algorithm: breaks integer factoring although not practical (requires millions of qubits)

Want: cryptosystems using quantum-resistant mathematical assumptions

Effort by NIST, post-quantum standardization

Generally, we use a polynomial size quantum circuit to break a (classical) mathematical assumption

Concern of "harvest now, decrypt later" (revealing ciphertexts may be risky)

1.1.2. Note: Grading

Grading will be based on class participation, assignments, and either scribing lecture notes OR research report

1.1.3. Note

Examples of post-quantum cryptographic assumptions include SIS (short integer solutions), and LWE (learning with errors)

These are lattice-based ("hard" problems in the classical sense which use lattices)

1.1.4. Concept: Short Integer Solutions

Suppose that $A \in \mathbb{Z}_q^{n \times n}$ where A is full rank

Goal: find $x \in \mathbb{Z}^n$ such that $Ax = 0$

Answer: set $x := 0 \in \mathbb{Z}^n$ where $\mathbb{Z}^n$ denotes n -dimensional column vectors

If we require that $x \neq 0$ then x cannot exist (full ranks implies invertibility / violates linear independence)

Suppose that $A \in \mathbb{Z}_q^{n \times m}$ where $m \gg n$

We split A in the following way: $A = [\bar{A} \mid A']$ where $\bar{A}$ is $n \times n$ and A' is the rest of A

Consider the following cases:

Case 1: the $n + 1$ th column is all zero, then $x = \begin{pmatrix} 0 \\ 0 \\ \vdots \\ 1 \\ \vdots \\ 0 \\ 0 \end{pmatrix}$ where 1 is in the $n + 1$ th position

Case 2: $\bar{A}$ is full rank and the $n + 1$ th column is non-zero. Then $x = \begin{pmatrix} u \\ -1 \\ 0 \\ \vdots \\ 0 \end{pmatrix}$ where $u \in \mathbb{Z}^n$ and the $n + 1$ th column is some $v \in \mathbb{Z}^n$ and such that $\bar{A}u = v$

Case 3: $\bar{A}$ is not full rank. Then $\exists u$ s.t. $\bar{A}u = 0$ and $u \neq 0$. Then $x = \begin{pmatrix} u \\ 0 \\ \vdots \\ 0 \end{pmatrix}$

In fact, if $m \gg n$ then there are many potential solutions. We are interested in finding B -short solutions.

1.1.5. Definition: B short solutions

We say that $u \in \mathbb{Z}^m$ is B short if $\|u\|_\infty \leq B$ where $\|\cdot\|_\infty$ is the infinity norm: $\max_i \{|u_i|\}$

1.2. Short Integer Solutions

1.2.1. Definition: $\text{SIS}_{n,m,B,q}$

given $A \in \mathbb{Z}_q^{n \times m}$,

find $x \in \mathbb{Z}^m$ s.t. $Ax = 0$; $x \neq 0$; $\|x\|_\infty \leq B$

it is believed that any PPT adversary $\mathcal{A}$ can find x only with negligible probability

1.2.2. Proposition: existence of B short solutions

Claim: if $m \geq 2n \log_2(q)$ then $\exists B$ short solutions

Proof:

Note that the number of possible B short vectors of dimension m is at most $(2B + 1)^m$ and the number of possible n dimensional vectors over $\mathbb{Z}_q$ is q^n , then

$$\begin{aligned} (2B + 1)^m &< q^n \\ m \log(2B + 1) &< n \log(q) \\ m &< \left(\frac{n \log(q)}{\log(2B + 1)} \right) \end{aligned}$$

which is a contradiction

(therefore there exists a solution)

□

1.2.3. Definition: Collision resistant hash functions (CRHF)

For each $h_k \in H = \{h_k : \{0, 1\}^m \rightarrow \{0, 1\}^n\}$ where $m \gg n$ it is hard to find x, y such that $x \neq y$ and $h_k(x) = h_k(y)$ and $k \xleftarrow{\$} \{0, 1\}^l$ where l is polynomial length (sometimes in the security parameter λ)

1.2.4. Theorem: Collision Resistant hash function using SIS assumption

The following hash function is collision resistant:

$$h_A(x) = Ax \text{ where } x \in \{0, 1\}^m \text{ and } A \in \mathbb{Z}_q^{n \times m}$$

Proof:

Suppose that i.e. if $\mathcal{A}$ knows h_A then it can output x, y such that $x \neq y$ and $h_A(x) = h_A(y)$

Then we construct $\mathcal{B}$ using $\mathcal{A}$ such that $\mathcal{B}$ breaks $\text{SIS}_{n,m,B,q}$; specifically,

$\mathcal{B}(A)$:

1. run $\mathcal{A}(h_A) \rightarrow (x, y)$ and output (x, y)

This satisfies all conditions to “break” “SIS” since

$$h_A(x) = h_A(y) \Rightarrow A(x - y) = 0$$

$$x \neq y \Rightarrow x - y \neq 0$$

$$\|x - y\|_\infty = 1$$

(note that x, y are 0/1 vectors)

□

1.2.5. Definition: inhomogeneous SIS, (ISIS)

It is “hard” for an adversary to find x such that $Ax = y$ given $y \xleftarrow{\$} \mathbb{Z}_q^m$ and A

1.2.6. Note

ISIS combined with trapdoors can be used to construct digital signature schemes

1.2.7. Definition: Trapdoor

Let $A \in \mathbb{Z}_q^{n \times m}$ then a trapdoor T is such that $AT = 0$ and each column of T is a B short solution. Formally we defined the polynomial time algorithm

$\text{TrapGen}(1^n, 1^m, 1^q)$:

outputs (A, T) subject to the above conditions. Additionally, T is full rank over $\mathbb{R}$

1.2.8. Note: Why over $\mathbb{R}$ specifically

Consider $M = \begin{pmatrix} q+1 & 1 \\ q & 0 \end{pmatrix}$ then $\det(M) = 0 \bmod q$ but $\det(M) \neq 0$ over $\mathbb{R}$

2. Lecture 2 Digital Signatures from I-SIS

2.0.1. Note: Recap: SIS and I-SIS

The following is a hard problem: given $A \in \mathbb{Z}_q^{n \times m}$ where $m \gg n$ and $m \approx O(n \log(q))$ find a short and non-trivial solution x in the kernel of A i.e. $Ax = 0$ (SIS)

SIS is conjectured to be secure against both classical PPT adversaries and quantum adversaries

Inhomogeneous SIS: Given (A, y) find short x such that $Ax = y$

2.0.2. Note: Trapdoors and TrapGen

The following algorithm exists and runs in efficient(i.e. polynomial) time:

$\text{TrapGen}(1^n, 1^m, 1^q, 1^B)$: outputs (A, T) such that $A \in \mathbb{Z}_q^{n \times m}$ and $T \in \mathbb{Z}^{m \times n}$ where the columns of T are each B short in the kernel of A , $AT = 0$ and T is full rank over $\mathbb{R}$

2.0.3. Definition: statistical distance (denoted approx_s) (??)

for a sample space χ we say that distributions $D_1 \approx_s D_2$ if

$$\frac{1}{2} \sum_{x \in \chi} |\Pr[x \leftarrow D_1] - \Pr[x \leftarrow D_2]| \leq \varepsilon$$

2.0.4. Note: A nice property 1 about TrapGen

$D_1 \approx_s D_2$ where $D_1 = \{A; (A, T) \leftarrow \text{TrapGen}()\}$ and $D_2 = \left\{A; A \xleftarrow{\$} \mathbb{Z}_q^{n \times m}\right\}$

2.0.5. Note

TrapGen does not violate SIS since in SIS matrix A is fixed

2.0.6. Note: A nice property 2 about TrapGen

given (A, T, y) there exists an algorithm to find short x such that $Ax = y$ called $\text{PREIMAGESAMP}(A, T, y) \rightarrow x$

Note that this does not violate I-SIS since the algorithm requires T as an input

2.0.7. Note: A nice property 3 about TrapGen

Consider D_1 and D_2 where

D_1 : sample $A \xleftarrow{\$} \mathbb{Z}_q^{n \times m}$ and generate short x and set $y := Ax$ then output (A, x, y)

D_2 : sample $(A, T) \leftarrow \text{TrapGen}$ and $y \xleftarrow{\$} \mathbb{Z}_q^n$ and find short x using (A, T) such that $Ax = y$ and output (A, x, y)
(remember that we can do this since PREIMAGESAMP exists)

2.0.8. Definition: Digital Signatures

setup: signing key sk and verification key vk

$\text{sign}(sk, D)$ generates signature σ on message D

$\text{verify}(vk, D, \sigma)$ generates VALID iff σ is valid on D

correctness: verify works security: for any PPT $\mathcal{A}$ which queries q signatures and outputs

$$\mathcal{A}((D_1, \sigma_1), \dots, (D_q, \sigma_q)) \rightarrow (D^*, \sigma^*)$$

the following is true:

$$\Pr[D^* \notin \{D_1, \dots, D_q\} \wedge (D^*, \sigma^*) \text{ is VALID}] \leq \text{negl}$$

(typically negligible in some security parameter used in the generation algo) (note that signing key should not be revealed)

2.0.9. Concept: Digital Signature scheme from I-SIS

in this case, out verification key $vk = A$ (the matrix) and the signing key sk is T (the trapdoor)

To sign some message D we can perform the following steps:

1. from D deterministically derive $y \in \mathbb{Z}_q^n$ (shown below)
2. find short x such that $Ax = y$
3. output $\sigma = x$

To verify a that σ is a valid signature on D :

1. check if $Ax = y$ and that x is short (output VALID/INVALID accordingly)

2.0.10. Definition: Formal Description of digital signature scheme (??)

note that in the below H is a hash function $H : \{0, 1\}^* \rightarrow \mathbb{Z}_q^*$

setup: run $\text{TrapGen}(1^n, 1^m, 1^q, 1^B \rightarrow (A, T))$ and output $vk = A$ and $sk = T$

$\text{sign}(sk, D)$:

1. set $y := H(D)$ and run $\text{PREIMAGESAMP}(A, T, y) \rightarrow x$ and output $x = \sigma$

$\text{verify}(vk, D, \sigma)$:

1. set $y := H(D)$ and check that $A\sigma = y$ and that σ is short (output VALID/INVALID accordingly)

Correctness: trivially true

Security: (use random oracle model)

3. Lecture 3: LWE and LWR

3.0.1. Note: Last time

Set the following: $A \in \mathbb{Z}_q^{m \times n}$, $s \in \mathbb{Z}_q^n$, $e \in \mathbb{Z}_q^m$

If we are given A and As , we can solve for s using gaussian elimination. Note that since $\mathbb{Z}_q$ is a field, all elements have an inverse.

However, given $As + e$ and A it is conjecture to be “hard” to solve for s

Note also that we let $\|e\|_\infty$ be small (say in $\text{poly}(\log(q))$)

3.0.2. Definition: SearchLWE(n,m,q, χ)

Let χ be some distribution, and $A \xleftarrow{\$} \mathbb{Z}_q^{m \times n}$, $s \xleftarrow{\$} \mathbb{Z}_q^n$, $e \leftarrow \chi^m$ then solving for s given $As + e$ is conjectured to be not possible any PPT algorithm as well as quantum adversaries.

3.0.3. Note

If χ^m is replaced with uniform distribution over $\mathbb{Z}^m$ then $As + e$ is uniformly random, therefore $As + e$ does not depend on s and therefore the probability of finding s is

$$\frac{1}{q^n}$$

3.0.4. Definition: DecisionLWE(n,m,q, χ)

let $(A, As + e)$ be as above and denote this distribution D_1

also, let the distribution D_2 be (A, b) where $b \xleftarrow{\$} \mathbb{Z}_q^m$ (recall that A is also uniformly sampled)

Then the following is true: $D_1 \approx_c D_2$ for both PPT and quantum adversaries

If $s \leftarrow \chi^m$ then the above still holds

3.0.5. Definition: public key encryption

Say that Alice wants to communicate with Bob, then Alice will send $\text{Enc}(x, \text{pk})$ to Bob using Bob's pk and Bob will decrypt the message with his sk

The algorithms are $\text{setup} \rightarrow (\text{pk}, \text{sk})$ $\text{Enc}(\text{pk}, x) \rightarrow C$ and $\text{Dec}(C, \text{sk}) \rightarrow x'$

Correctness: $x = x'$

Security: $\{\text{Enc}(\text{pk}, x_0)\} \approx_c \{\text{Enc}(\text{pk}, x_1)\}$

(recall the chose a random bit game)

3.0.6. Definition: Oded Regev public key encryption scheme

Setup: sample $A \xleftarrow{\$} \mathbb{Z}_q^{m \times n}, s \xleftarrow{\$} \mathbb{Z}_q^n, e \leftarrow \chi^m$

let the public key pk be $\bar{A} = [A \mid As + e]$ of size $(m \times n + 1)$

let the secret key sk be s

Enc(pk, $x \in \{0, 1\}$): (one bit)

Sample $r \xleftarrow{\$} \{0, 1\}^{1 \times m}$

$$C = \begin{cases} r\bar{A} & \text{if } x = 0 \\ r\bar{A} + \begin{pmatrix} 0 \\ \vdots \\ 0 \\ \lfloor \frac{q}{2} \rfloor \end{pmatrix}^T & \text{if } x = 1 \end{cases}$$

Dec(sk, C):

compute

$$\begin{aligned} C \cdot \begin{pmatrix} -s \\ 1 \end{pmatrix} &= \langle r\bar{A} + x(0 \dots 0 \lfloor \frac{q}{2} \rfloor), \begin{pmatrix} -s \\ 1 \end{pmatrix} \rangle \\ &= r\bar{A} \cdot \begin{pmatrix} -s \\ 1 \end{pmatrix} + x(0 \dots 0 \lfloor \frac{q}{2} \rfloor) \cdot \begin{pmatrix} -s \\ 1 \end{pmatrix} \\ &= r \cdot e + x \lfloor \frac{q}{2} \rfloor \end{aligned}$$

We can note that some typical parameters ranges are $q = 2^\lambda, n = \text{poly}(\log(q)), m = n \log(q), e = \text{poly}(\log(q))$

Therefore we can reason that $\|r \cdot e\|_\infty \leq m \cdot \|e\|_\infty \ll q$

And so the decrypted bit will be $x' = 0$ if $C \cdot \begin{pmatrix} -s \\ 1 \end{pmatrix} \leq \frac{q}{4}$ and $x' = 1$ otherwise

(recall the interval in the notes)

3.0.7. Proposition: Security of Definition 3.0.6

3.0.6 satisfied the security definition for general public key encryption in 3.0.6

Proof:

The proof uses the hybrid argument

Let $H_1 = \text{Enc}(\text{pk}, 0)$ and $H_2 = r \cdot [A \mid b]$ where $b \stackrel{\$}{\leftarrow} \mathbb{Z}_q^m$ then $H_1 \approx_c H_2$ by the DecisionLWE assumption and the randomness of $r \stackrel{\$}{\leftarrow} \{0, 1\}^{1 \times m}$

Let $H_3 = C \stackrel{\$}{\leftarrow} \mathbb{Z}_q^{1 \times (n+1)}$ by the fact $\{(B, rB)\} \approx_c \{B, u\}$ where B is size $m \times n$ and u is a random vector

Here we can set $B := [A \mid b]$ and $u := r$

Let $H_4 = C = C' + (0 \dots 0 \lfloor \frac{q}{2} \rfloor)$ where $C' \stackrel{\$}{\leftarrow} \mathbb{Z}_q^{1 \times (n+1)}$ then $H_3 \approx_c H_4$ by the randomness of C'

Let $H_5 = C = r \cdot [\bar{A} \mid b] + (0 \dots 0 \lfloor \frac{q}{2} \rfloor)$ and $H_6 = \text{Enc}(\text{pk}, 1)$ then $H_4 \approx_c H_5 \approx_c H_6$ by the randomness of r and the definition of $\text{Enc}(\text{pk}, 1)$

□

3.0.8. Definition: dual regev public key encryption scheme

Setup $\rightarrow (A, T) \leftarrow \text{TrapGen}$

here we have $TA = 0$ and $A \in \mathbb{Z}_q^{m \times n}$ and $T \in \mathbb{Z}^{m \times m}$ (we can note that $TA \stackrel{?}{=} 0$ (??))

Anways the public key is A and the secret key is T

Encryption on a message x outputs $C = As + e + x(0 \dots 0 \lfloor \frac{q}{2} \rfloor)^T$

Decryption: compute $T(As + e + x(0 \dots 0 \lfloor \frac{q}{2} \rfloor)^T) = TAs + Te + Tx(0 \dots 0 \lfloor \frac{q}{2} \rfloor)^T = 0 + \text{small} + (\cdot)$

So the decrypted bit will be 0 if the above value is approx 0 and 1 if $\|(\text{above})\|_\infty \geq \frac{q}{4}$

Correctness follows

Security follows a similar path to proposition 3.0.7

3.1. LWR

3.1.1. Note: Intuition

recall that $As + e \bmod q$ is indistinguishable from uniformly random suppose $q = 2^\lambda$ and χ be uniform over $\{0, \dots, 2^l\}$ such that $l \ll \lambda$

consider the binary representation of $As + e$ then this implies that the least significant l bits are jumbled

Learning with rounding simply discards these bits

3.1.2. Definition: rounding function $\lfloor \cdot \rfloor : \mathbb{Z}_p \rightarrow \mathbb{Z}_q$

$$\lfloor \cdot \rfloor(x)_p = \frac{\bar{x}}{\frac{q}{p}} \bmod p$$

where $\bar{x} = x \bmod q$

divide into $\frac{q}{p}$ size blocks

an alternative definition is $x \frac{p}{q} \bmod p$

can round up or down

4. Lecture 4: LWR, PRF, reductions

4.0.1. Definition: LWR

$A \xleftarrow{\$} \mathbb{Z}_q^{m \times n}, s \xleftarrow{\$} \mathbb{Z}_q^n, u \xleftarrow{\$} \mathbb{Z}_p^m$ with $p \ll q$

Claim: $\{A, \lfloor As \rfloor_p\} \approx_c \{A, u\}$

We will use the floor function to denote the rounding

Note that the value rounded is at the i th block where each block is of size $\frac{q}{p}$ on the real line

Interpretation: $\lfloor As \rfloor_p$ drops least significant bits of As in the case that p, q are powers of 2

4.0.2. Note: Fact 1

$\lfloor As + e \rfloor_p = \lfloor As \rfloor_p$ with high probability

4.0.3. Note: Fact 2

$\{\lfloor u \rfloor_p\} \approx_c \{u'\}$ where $u \xleftarrow{\$} \mathbb{Z}_q^m, u' \xleftarrow{\$} \mathbb{Z}_p^m$

4.0.4. Definition: pseudorandom functions

A function $F : \{0, 1\}^\lambda \times \{0, 1\}^n \rightarrow \{0, 1\}^m$ is called pseudorandom if

$$\left| \Pr \left[1 \rightarrow \mathcal{A}^{F(k, \cdot)} : k \xleftarrow{\$} \{0, 1\}^\lambda \right] - \Pr \left[1 \leftarrow \mathcal{A}^{g(\cdot)} : g \xleftarrow{\$} \mathcal{F} \right] \right| \leq \text{negl}$$

where $\mathcal{F}$ denotes all functions from n bits to m bits

4.0.5. Note

$\text{PRF}(k, x) = \text{PRF}(k_1, x_1) \parallel \text{PRF}(k_2, x_2)$ is not a pseudorandom function

where the key and input are split in half respectively

This is due to F being deterministic

4.0.6. Notation

Typically, we write u to be an element coming from the uniform distribution

4.0.7. Definition: Construction 1 of PRF using LWR

$$\text{PRF}(k, x) = \lfloor \text{PRF}(k_1, x_1) \cdot \text{PRF}(k_2, x_2) \rfloor_p (\dagger)$$

Conceptually we can think of $k \in \{0, 1\}^{2\lambda}$ and $x \in \{0, 1\}^{2l}$

Let the output of $\text{PRF}(k_1, x_1)$ be A and let the output of $\text{PRF}(k_2, x_2)$ be s

Then from fact 2 from before we have that:

Case 1: fix x_1 then

$$\left\{ \left(A, \lfloor As_1 \rfloor_p, \dots, \lfloor As_t \rfloor_p \right) \right\} \approx_c \{ (A, u_1, \dots, u_t) \}$$

Case 2: fix x_2 then

$$\left\{ \left(A_1, \dots, A_t, \lfloor A_1 s \rfloor_p, \dots, \lfloor A_t s \rfloor_p \right) \right\} \approx_c \{ (A_1, \dots, A_t, u_1, \dots, u_t) \}$$

4.0.8. Theorem

$(\dagger)$ is a PRF

Proof:

Note the above implies that

$$\left\{ \left\lfloor \text{PRF}\left(k_1, x_1^{(1)}\right) \cdot \text{PRF}\left(k_2, x_2^{(1)}\right) \right\rfloor_p, \dots, \left\lfloor \text{PRF}\left(k_1, x_1^{(t)}\right) \cdot \text{PRF}\left(k_2, x_2^{(t)}\right) \right\rfloor_p \right\} \approx_c \{u_1, \dots, u_t\}$$

for queries $x^{(1)}, \dots, x^{(t)}$

□

4.0.9. Example: Recursive computation of 4.0.7 PRF

let the key be $k = \{S_{ib}\}$ for $i \in 1, \dots, l$ where l is the input length and $b \in \{0, 1\}$ and $S_{ib} \in \mathbb{Z}_{q_0}^{n \times n}$

Let $q_0 > q_1 > q_2 > \dots$ such that each q_i is “significantly” smaller than previous q_i (e.g. $2^\lambda, 2^{\frac{\lambda}{2}}, 2^{\frac{\lambda}{4}}$, etc.)

Let $x \in \{0, 1\}^8$ then $\text{PRF}_k(x) =$

$$\left\lfloor \left\lfloor \left\lfloor S_{1x_1} \cdot S_{2x_2} \right\rfloor_{q_1} \cdot \left\lfloor S_{3x_3} \cdot S_{4x_4} \right\rfloor_{q_1} \right\rfloor_{q_2} \cdot \left\lfloor \left\lfloor S_{5x_5} \cdot S_{6x_6} \right\rfloor_{q_1} \cdot \left\lfloor S_{7x_7} \cdot S_{8x_8} \right\rfloor_{q_1} \right\rfloor_{q_2} \right\rfloor_{q_3}$$

4.0.10. Definition: Construction 2 of PRF using LWR

Let the key be $k = \{(A \in \mathbb{Z}^{m \times n}, \{S_{ib}\}_{ib})\}$ where S_{ib}, i, b are the same as in 4.0.9

Then we define

$$\text{PRF}(k, x) = \left[A \cdot \prod_i S_{ix_i} \right]_p$$

We note the following for error matrices E_i (which has small entries) :

$$\left[A \cdot \prod_i S_{ix_i} \right]_p \approx_c \left[(A + E_i) \cdot \prod_i S_{ix_i} \right]_p$$

Then it follows that $\left[A \cdot \prod_{i=1} S_{1,x_i} \right]_p \approx_c \left[A \cdot \prod_{i=2} S_{2,x_i} \right]_p \approx_c \dots \approx_c [U]_p$ where $U \xleftarrow{\$} \mathbb{Z}_q^{m \times n}$

4.1. Reductions between SIS, LWE, LWR

We now show some reductions between SIS, LWE, and LWR. Note the use of contrapositives.

4.1.1. Definition: Easy to Solve

A problem is easy to solve if there exists some adversary (classical or quantum) which can solve the problem in polynomial (or potentially subexponential) time

4.1.2. Definition: SIS (search) is easy

For $A \in \mathbb{Z}_q^{m \times n}$ can efficiently find $x \in \mathbb{Z}_q^m$ such that $x^T A = 0$ where $x \neq 0, \|x\|_\infty = \text{poly}(\log(q)) \ll q$

4.1.3. Definition: Decision LWE is easy

can efficiently distinguish $\{(A, As + e)\}, \{(A, u)\}$

4.1.4. Definition: Search LWE is easy

given $(A, As + e)$ can efficiently find $s \in \mathbb{Z}_q^n$

4.1.5. Theorem

if SIS is easy then decision LWE is easy

Proof:

LWESolver(A, b):

1. $x \leftarrow \text{SIS solver}(A)$
2. If $|x^T b| < \frac{q}{4}$ then b is from LWE sample else from uniform

Case 1: $b = As + e$ then $x^T(As + e) = 0 + x^T e$ which is “small”

Case 2: b is uniform then with constant probability $x^T b$ is not small

□

4.1.6. Theorem

if search LWE is easy then decision LWE is easy

Proof:

dLWE Solver (A, b):

$s \leftarrow \text{sLWE solver}(A, b)$

if $\|b - As\|_\infty$ is small then b is from LWE else from uniform

case 1: $b = As + e$ then $\|b - As\|_\infty$ is always small

case 2: b is uniform (in the case that $m \gg n$)

note the following:

$$\Pr[b = As + e \text{ for any } s \in \mathbb{Z}_q^n, e \in [-B, B]^m] \leq \frac{1}{\text{poly}}$$

this is because the number of possibilities of $As + e$ are much smaller than the possibilities of b

i.e. $q^n \cdot (2B + 1)^m \ll q^m$ since $n \log(q) + m \log(2B + 1) \ll m \log(q)$ by $m \gg n$ and $B \ll q$

□

5. Lecture 5: more reductions, lattices

5.0.1. Theorem

if decision LWE is easy then search LWE is easy

Proof:

given $(A, As + e)$ the goal is to find s where $A \in \mathbb{Z}_q^{m \times n}$, $s \in \mathbb{Z}_q^n$, $e \in \mathbb{Z}_q^m$

write A in the following way (separate out leftmost column):

$$A = \begin{pmatrix} a_1 & \leftarrow a'_1 \rightarrow \\ \vdots & \vdots \\ a_m & \leftarrow a'_m \rightarrow \end{pmatrix}$$

then we can make a guess for $g \in \mathbb{Z}_q$ and sample $c_i \xleftarrow{\$} \mathbb{Z}_q$

Then the following is relevant:

$$As + e + gc = \begin{pmatrix} a_1 s_1 + \langle a'_1, s'_1 \rangle + e_1 \\ \vdots \\ a_m s_1 + \langle a'_m, s'_1 \rangle + e_m \end{pmatrix} + \begin{pmatrix} gc_1 \\ \dots \\ gc_m \end{pmatrix}$$

We slo define $A' := A + \begin{pmatrix} c_1 & 0 & \dots & 0 \\ \vdots & \vdots & \ddots & \vdots \\ c_m & 0 & \dots & 0 \end{pmatrix}$

We can compute $\alpha \leftarrow \text{dLWE}(A', As + e + gc)$

If α is “LWE” then the guess g is correct, else the guess g is incorrect

Consider the following cases:

Case 1: $g_1 = s_1$ (in general: $g_i = s_i$)

Then

$$As + e + gc = \begin{pmatrix} a_1 s_1 + \langle a'_1, s'_1 \rangle + e_1 + c_1 s_1 \\ \vdots \\ a_m s_1 + \langle a'_m, s'_1 \rangle + e_m \end{pmatrix} = \begin{pmatrix} (a_1 + c_1) s_1 + \langle a'_1, s'_1 \rangle + e_1 \\ \vdots \\ \end{pmatrix} = A' s + e$$

which is a LWE sample

Case 2: $g_1 \neq s_1$ (in general: $g_i \neq s_i$) then

$$As + e + gc$$

is uniform due to the uniformity of c

Note that we need to guess over all possible s to obtain the answer, but do not need to also check every possible e

□

5.0.2. Theorem: Whether s in LWE comes from uniform or chi (small) distribution doesn't matter

$$1: \{(A, As + e)\} \approx_c \{(A, u)\}$$

$$2: \{(A, As + e)\} \approx_c \{(A, u)\}$$

Theorem: if 2 is easy then 1 is easy

(if 1 is hard then 2 is hard)

Proof: ??

**5.0.3. Theorem: LWR is easy implies LWE is easy**

i.e. LWE is hard implies LWR is hard

Proof:

$$\text{LWR: } \{(A, \lfloor As \rfloor_p)\} \approx_c \{(A, u)\}$$

$$\text{LWE: } (A, As + e) \approx_c \{(A, u)\}$$

$$\text{Note that } \{(A, \lfloor As \rfloor_p)\} \approx_s \{(A, \lfloor As + e \rfloor_p)\} \approx_s \{(A, \lfloor u \rfloor_p)\} \approx_s \{(A, u)\}$$

$$\text{implies that } \{(A, \lfloor As + e \rfloor_p)\} \approx_s \{(A, \lfloor u \rfloor_p)\}$$



5.1. Lattices

5.1.1. Note

So far we have seen constructions of LWE, SIS_< and LWR as well as reductions between them, as well as a few applications of these assumptions

They rely on average case assumptions which are stronger than worst case assumptions

But why do they are conjectured to be post-quantum secure?

5.1.2. Definition: Lattices

A lattice $\mathcal{L}(u_1, \dots, u_l) = \{\sum s_i u_i : s_i \in \mathbb{Z}\}$ where $u_1, \dots, u_l \in \mathbb{R}^n$ are linearly independent (called the basis)

$\mathcal{L}$ has rank l and is a discrete additive subgroup of $\mathbb{R}^n$ where n is the dimension of $\mathcal{L}$ (note that $n \geq l$)

For example $L((0, 1), (1, 0))$ is a lattice and it has infinitely many bases

5.1.3. Definition: Lattice basis characterization

Let B_1, B_2 be two matrices whose columns are two different bases of some lattice $\mathcal{L}$

Then $B_1 U = B_2$ where $\det(U) = 1$

U is called “unimodular”

5.1.4. Definition: Fundamental Parallelopiped

We denote the fundamental Parallelopiped as $\mathcal{P}(u_1, \dots, u_l) = \{s_i u_i : s_i \in \mathbb{R}; 0 \leq s_i \leq 1\}$

Then $u_1, \dots, u_l$ is a lattice iff $\mathcal{P}(u_1, \dots, u_l) \cap \mathcal{L}(u_1, \dots, u_l) = \{0\}$

5.1.5. Definition: shortest vector in lattice

Defined by

$$\lambda_1(\mathcal{L}) = \min_{v \in \mathcal{L} \setminus \{0\}} \|v\|_2$$

where the norm is a euclidean 2 norm

5.1.6. Definition: $\lambda_i(\mathcal{L})$

We define $\lambda_i(\mathcal{L}) :=$ smallest r such that some subset of $\mathcal{L}$ contains i l.i. vectors of norm $\leq r$

5.1.7. Note

λ_1, λ_i are relative to the origin

A “good lattice basis” means that this basis generates a lattice for which calculating λ_1 is easy

A “bad lattice basis”: cannot efficiently calculate λ_1

5.1.8. Definition: Coset of a lattice

Let $\mathcal{L}$ be a lattice then coset of $\mathcal{L} = \{c + \mathcal{L} : v \in \mathcal{L}\}$ for some $c \in \mathbb{R}^n$

Note that if $c \in \mathcal{L}$ then $c + \mathcal{L} = \mathcal{L}$

5.1.9. Definition: Dual Lattice

The dual lattice is denoted as $\mathcal{L}^* = \{w : \langle w, v \rangle \in \mathbb{Z} \text{ where } w \in \mathbb{Z} \text{ and all } v \in \mathcal{L}\}$

5.1.10. Definition: Shorted vector problem

given basis B of an n dimensional lattice find $v \in \mathcal{L}$ such that $\|v\| \leq \gamma(n) \cdot \lambda_1(\mathcal{L})$

If B contains the shortest vector then easy but B can be any basis

5.1.11. Definition: GapSVP $_\gamma$ / ApproximateSVP $_\gamma$

given basis B of an n -dim lattice find $v \in \mathcal{L}$ such that $\|v\| \leq \gamma(n) \cdot \lambda_1(\mathcal{L})$

The smaller the γ the harder this problem is to solve

In fact we believe the following results hold:

$$1 \leq \gamma \leq 2^{\log(n)^{1-\varepsilon}} : \text{for some } \varepsilon \text{ this is NP-Hard}$$

$$2^{\log(n)^{1-\varepsilon}} \leq \gamma \leq n^{1.5} : \text{this is in } \text{NP} \cap \text{coNP}$$

$$n^{1.5} \leq \gamma \leq 2^{n^\delta} \text{ for some } \delta < 1 \text{ assumptions is used to build cryptosystems}$$

$$\gamma \leq 2^{o(n)} \text{ exists LLL poly-time algo to solve}$$

5.1.12. Theorem: Hardness of GapSVP $_\gamma \Rightarrow$ hardness of SIS $_{n,m,q,B}$ (intuition)

$$m = \text{poly}(n)$$

Here the setting is $q \geq B \cdot \text{poly}(n)$

and $\gamma = B \cdot \text{poly}(n)$

Proof:

(intuitive idea of proof): use contradiction / contrapositive

Consider a random lattice point v and some i perturbations such that $u_i = v + e_i$ which generate $u_1, \dots, u_n$

Also, consider $\begin{pmatrix} \uparrow & \uparrow & \dots & \uparrow \\ u_1 & u_2 & \dots & u_n \\ \downarrow & \downarrow & \dots & \downarrow \end{pmatrix} \begin{pmatrix} r_1 \\ \vdots \\ r_n \end{pmatrix} = 0$ as in the SIS problem

our goal is to find $r_i \in \mathbb{Z}$ such that $\sum_i r_i u_i = 0$ and at least one of $r_i \neq 0$

In fact by our assumption we can find $r_1, \dots, r_n$ and

$$0 = \sum r_i u_i = \sum r_i v + \sum r_i e_i = 0$$

Since $\sum r_i v \in \mathcal{L} \wedge 0 \in \mathcal{L}$ then this implies that $\sum r_i e_i \in \mathcal{L}$ where e_i are both r_i, e_i are small (roughly)

So $\sum r_i e_i \in \mathcal{L}$ is a solution to GAPSVP $_\gamma$

□

5.1.13. Definition: Discrete Gaussian function over $\mathbb{R}^n$

$$\rho_s(x) = e^{(-\pi \|x\|^2)/s^2}$$

where $x \in \mathbb{R}^n$

The standard discrete gaussian is when $s = 1$

The continuous gaussian is derived by integrating ρ_s and is defined by

$$\frac{\rho_s(x)}{s^n}$$

5.1.14. Definition: Discrete Gaussian distribution

We parametrize this distribution with lattice $\mathcal{L} \subseteq \mathbb{R}^n$ and “width” s as well as $c \in \mathbb{R}^n$ and define

$$\mathcal{D}_{c,\mathcal{L},s} = \begin{cases} \rho_s(x) & \text{if } x \in c + \mathcal{L} \\ 0 & \text{otherwise} \end{cases}$$

For example, if $c = 0$ then points further away from the origin have less likelihood of being sampled

5.1.15. Problem 1

Given basis B which generates $\mathcal{L}$ how to (efficiently) sample from $\mathcal{D}_{c,\mathcal{L},s}$?

Solution

Not really given in the lecture

However it was mentioned that setting s to “larger” values creates a distribution that is more “spread out or closer to uniform” (making it easier to sample from)

For sufficiently small values of s we cannot efficiently sample by the GapSVP_γ problem

5.1.16. Theorem

Hardness of $\text{GapSVP}_\gamma \Rightarrow$ Hardness of decision $\text{LWE}_{n,m,q,\chi}$ (rough idea)

Proof:

We set the following parametrization: $m = \text{poly}(n)$; $q \leq 2^{n^{0.99}}$; $\gamma = O\left(\frac{n}{\alpha}\right)$, $s = \alpha q > \sqrt{2n}$

The general concept is to keep sampling from $\mathcal{D}_{\mathcal{L},s_i}$ where smaller and smaller values of s_i which is possible by assumption

We went over a quantum algorithm which is considerably complex

□

6. Lecture 6

6.0.1. Note

Lecture 6 covered 5.1.5 to 5.1.16

7. Lecture 7 (online)

7.0.1. Note

Monday's lecture: https://drive.google.com/file/d/1rXsNm-CpNMaq1TxN3Le-dTIffv7_Tf1A/view?usp=drive_link

8. Lecture 8 (youtube)

8.0.1. Note

Wednesday's lecture: https://www.youtube.com/watch?v=Lo-_ZBqGa7I&t=3790s (this lecture is by Chris Peikert who is one of the experts on lattices in general and ring LWE in particular)

9. Lecture 9 - Insecure Ring-SIS

9.0.1. Note: Recap

Topics recalled: SIS, decision LWE, search LWE hard implies decision LWE hard, LWE is equally hard whether $s \xleftarrow{\$} \mathbb{Z}_q^n$ or $s \xleftarrow{\$} \chi^n$, SIS hardness and LWE hardness via computational problems over lattices (GAPSVP_γ)

Additionally, recall that the distribution χ can be thought of as a discrete gaussian distribution

We also recalled a SIS-based hash function where $A \in \mathbb{Z}_q^{n \times m}$ and $h_A(x) = Ax$ where $x \in \mathbb{Z}_2^m$

If $m = O(n \log(q))$ then the storage cost (in bits) will be $mn \log(q) = n^2 \text{poly}(\log(q))$

and the computational cost will be $mn \log(q) = n^2 \text{poly}(\log(q))$ since finite field addition and multiplication typically take $\text{poly}(\log(q))$ time

our goal will be to define a more efficient hash function

9.0.2. Definition: Structured matrices $X, X^2, \dots$ generate a lattice which is isomorphic to univariate polynomial quotient rings

Suppose that $A \in \mathbb{Z}_q^{n \times m}$ and that the first column of A is $\begin{pmatrix} a_1 \\ \vdots \\ a_n \end{pmatrix} \xleftarrow{\$} \mathbb{Z}_q^n$

We define A pictorially: $A = \begin{pmatrix} a_1 & a_n & a_{n-1} & a_{n-2} & \dots \\ \vdots & a_1 & a_n & a_{n-1} & \dots \\ \vdots & \vdots & a_1 & a_n & \dots \\ \vdots & \vdots & \vdots & a_1 & \dots \\ \vdots & \vdots & \vdots & \vdots & \dots \\ a_n & a_{n-1} & a_{n-2} & a_{n-3} & \dots \end{pmatrix}$

As we can see the columns of A are generated by permutations of the first column, which means that A can have storage cost $n \log(q)$ (A can be reduced to a “short description”)

Observe also that the first column of $A_1 = I \begin{pmatrix} a_1 \\ \vdots \\ a_n \end{pmatrix}$

Second column is $A_2 = \begin{pmatrix} 0 & \dots & 0 & 1 \\ 1 & \dots & 0 & 0 \\ 0 & 1 & \dots & 0 \\ 0 & 0 & \ddots & 0 \\ 0 & 0 & \dots & 1 \end{pmatrix} = X \begin{pmatrix} a_1 \\ \vdots \\ a_n \end{pmatrix}$

Similarly, the third column of A is $X^2 \begin{pmatrix} a_1 \\ \vdots \\ a_n \end{pmatrix}$ and the i th column of A is $X^{i-1} \begin{pmatrix} a_1 \\ \vdots \\ a_n \end{pmatrix}$

In fact, we can write A as $a_1 I + a_2 X + a_3 X^2 + \dots + a_n X^{n-1}$

Then can define $I, X^1, X^2, \dots, X^{n-1}$ to be a basis which generates the lattice $\mathcal{L}_{\text{str}} = \left\{ \sum_{i=1}^n a_i X^{i-1} : a_i \in \mathbb{Z} \right\}$

This means $A \in \mathcal{L}_{\text{str}}$ and we can create an isomorphism from $\mathcal{L}_{\text{str}} \leftrightarrow \mathbb{Z}[x]$ (the univariate polynomials with coefficients from $\mathbb{Z}$)

$\mathbb{Z}[X]$ forms a ring, allowing us to inherit some nice algebraic properties (of rings)

However, the above isomorphism is not entirely correct. Since $X^n = I$ we have $x^n = 1 \Rightarrow x^n - 1 \equiv 0$ so we will need to modulo by $x^n - 1$ to form the quotient ring $\mathbb{Z}[x]/(x^n - 1) = \left\{ \sum_{i=0}^{n-1} \alpha_i x^i : \alpha_i \in \mathbb{Z} \right\}$ (these are the remainders of $\mathbb{Z}[x]$ when divided by $x^n - 1$)

the isomorphism is the map $\phi(X^i) = x^i$

9.0.3. Notation

We denote $\mathbb{Z}[x]/(x^{n-1}, q)$ to be the set of all degree d polynomials with coefficients in $\mathbb{Z}_q$ and the polynomials are modulo x^{n-1}

9.0.4. Example

Working modulo $x^n - 1$ we have $x^{n+1} = x(x^n - 1) + x \equiv x$

9.0.5. Definition: New Hash Function

We modify our original hash function $h_A(y) = Ay$ to instead be $h_A(M_y) = AM_y$ where M_y is size $m \times m$ and has short entries, and in addition $A, M_y \in \mathcal{L}_{\text{str}}$

Further we would like to work with polynomials, so we map $A \leftrightarrow P_A$ and $M \leftrightarrow P_M$ and the definition of the hash function becomes $h_A(M) = (P_A)(P_M) \bmod(q, x^n - 1)$

Then the runtime is $n^2 \text{poly}(\log(q))$ with naive algorithm but $n \text{poly}(\log(q))$ using FFT

We can also define $P_A^1 \dots P_A^l$ to be the key of hash function then we can have hash function definition $\sum_{i=1}^l P_A^i P_M^i$ where the P_M^i again have short entries and the runtime will still be $n \text{poly}(\log(q))$ if $l = \text{poly}(\log(q))$

The security of this hash function follows as long as ring SIS holds and we pick the right quotient ring

9.0.6. Definition: Ring SIS (insecure version)

Let $p_1, \dots, p_l \xleftarrow{\$} \mathbb{Z}_q[x]/x^n - 1$ and $q_1, \dots, q_l \leftarrow \chi$

We denote χ to be a distribution over $\mathbb{Z}_q[x]/x^n - 1$ where the coefficients are small

Problem: given $(p_1, \dots, p_l, r)$ find $q_1, \dots, q_l$ such that $\sum p_i q_i = r \bmod(q, x^n - 1)$

9.0.7. Theorem

Definition 9.0.6 is not secure

Proof:

Intuition: $x^n - 1$ is factorable

9.0.8. Lemma: high chance of factorability of

pick a random p from $\mathbb{Z}_q[x]/q, x^n - 1$ then the probability that $(x - 1)$ divides p is $\geq 1/q$

Proof:

(roughly)

$$\text{if } p = (x - 1) \left(\sum_{i=0}^{n-1} \alpha_i x^i \right) = (x - 1)(\alpha_0 + \alpha_1 x_1 + \dots + \alpha_{n-1} x^{n-1}) = (\alpha_{n-1} - \alpha_0) + (\alpha_1 - \alpha_2)x + (\alpha_2 - \alpha_3)x^2 + \dots$$

Then the probability that the sum of coefficients of p is $0 \bmod q$ is $1/q$

□

Note that if we pick a random p as in the above lemma then

$$p(1 + x + x^2 + \dots + x^{n-1}) = q(x - 1)(1 + x + x^2 + \dots + x^{n-1}) = 0 \bmod (q, x^n - 1)$$

Then the following attack is possible when $r = 0$:

Given $p_1, \dots, p_l$ find one p_i such that $(x - 1)$ divides p_i

Set $q_i = 1 + x + x^2 + \dots + x^{n-1}$

Set $q_j = 0 \forall j \neq i$

□

10. Lecture 10: ring LWE priv/pub encryption, code based assumptions and error correcting codes

10.0.1. Definition: Rings SIS (“regular”)

find $q_1, \dots, q_l$ such that $\sum p_i q_i = 0$ given $p_1, \dots, p_l$ where $q_1, \dots, q_l$ must have small coefficients

10.0.2. Note

In fact the hash function from 9.0.5 is secure even if only regular SIS is secure

If the hash function is not secure then an efficient adversary $\mathcal{A}$ can find $q_1, \dots, q_l$ and $q'_1, \dots, q'_l$ that both multiply to with the p 's to equal r

Then a solution to ring SIS regular would be $q_1 - q'_1, \dots, q_l - q'_l$

10.0.3. Definition: lattice generate by $\mathbb{Z}_q[x]/x^n + 1$

We define the matrix $X' = \begin{pmatrix} 0 & 0 & \dots & 0 & -1 \\ 1 & 0 & \dots & 0 & 0 \\ 0 & 1 & \dots & 0 & 0 \\ \vdots & 0 & \ddots & 0 & 0 \\ \vdots & 0 & \dots & 1 & 0 \end{pmatrix}$

and the new lattice $\mathcal{L}'_{\text{str}}$ to be generated by basis $I, X', (X')^2, \dots, (X')^{n-1}$

10.0.4. Definition: Ring LWE

$a_1, \dots, a_l \xleftarrow{\$} \mathbb{Z}_q[x]/x^n + 1$

set $b_1, \dots, b_l$ where $b_i := a_i s + e_i \bmod (q, x^n + 1)$

where $s \leftarrow \chi, e_i \leftarrow \chi$ where χ is a distribution over polynomials with small coefficients in $\mathbb{Z}_q[x]/x^n + 1$

10.0.5. Definition: Private Key Enc using ring LWE

Gen: $s \leftarrow \chi$

Enc($s, \mu \in \{0, 1\}$) :

if $\mu = 0$ then output $(a, as + e)$ where a, e as above

if $\mu = 1$ then output $(a, \text{uniform})$

Dec(s, c) :

parse $c = (a, b)$

compute $b - as \bmod (q, x^n + 1)$

if all coefficients of $b - as$ are small then output $\mu = 0$ else output $\mu = 1$

security follows by ring LWE hardness

10.0.6. Definition: Public Key Enc using ring LWE

Gen: $s \leftarrow \chi, a' \leftarrow \mathbb{Z}_q[x]/x^n + 1, e \leftarrow \chi, y = a's + e \bmod (q, x^n + 1)$

set public key to be (a', y) and secret key to be s

Enc: $((a', y), \mu \in \{0, 1\})$:

1. $r \leftarrow \chi, e' \leftarrow \chi, e'' \leftarrow \chi$
2. set $a := a'r + e' \bmod (q, x^n + 1)$
3. set $b := yr + e'' + \lfloor \frac{q}{2} \rfloor \mu \bmod (q, x^n + 1)$
4. output (a, b)

Dec: $(s, (a, b))$

Observe that $b = (a's + e)r + e'' + \lfloor \frac{q}{2} \rfloor \mu = a'rs + er + e'' + \lfloor \frac{q}{2} \rfloor \mu$

$as = a'(rs) + (e's)$

then $b - as = \text{small} + \lfloor \frac{q}{2} \rfloor \mu \bmod (q, x^n + 1)$

This implies if all coefficient of $b - as$ small then $\mu = 0$ else $\mu = 1$

Security: invoke ring LWE assumption

10.0.7. Definition: prisoner's game

7 prisoners are each assigned a hat with half probability each of being red or blue

prisoners can guess if they are wearing red, blue, pass

win if nobody answers wrong AND at least one prisoner doesn't answer pass

10.1. error correcting codes

(skip)

11. Lecture 11: Learning Parity with noise/ error correcting codes, Peikert

(skip due to midterm)

12. Lecture 12: LPN based hash functions, cryptographic group actions

12.0.1. Definition: Hash function (LPN)

We defined a collision resistant hash function using a expand then compress approach

Conceptually, we take a vector and “sparsify” it using a function $\text{expand}(y) \rightarrow z$ where y, z are binary vectors

If n is a security parameter then we divide y into blocks of size $L = o(\log(n))$ and expand each of these blocks into a corresponding block of z of size 2^L . The result is the expansion of the size of y , and both y and z have the same number of blocks

Specifically, each block in z is a “one-hot” vector with a 1 in position i where i is the integer representation of the corresponding block in y

Then, for a hash key $M \in \mathbb{Z}_2^{n \times q}$ we define the hash function

$$h_M(x) = M \cdot \text{expand}(x)$$

where $q > k > n, x \in \mathbb{Z}_2^k, \text{expand}(x) \in \mathbb{Z}_2^q$

Recall that hash functions are typically length reducing

12.0.2. Proposition: Security of 12.0.1

We claim that $\{h_M\}$ is collision resistant

Proof:

Suppose on the contrary that $\{h_M\}$ is not collision resistant

Then there exists an efficient adversary $\mathcal{A}$ such that $\mathcal{A}(h_M) \rightarrow (y, y')$ such that $y \neq y'$ and

$$h_M(y) = h_M(y') \Rightarrow M \cdot \text{expand}(y) - \text{expand}(y') = 0$$

Goal: design a PPT $\mathcal{B}$ which distinguishes $\{M^T, sM^T + e\}$ and $\{(M^T, u)\}$ (recall the definition of decision LPN)

Let v be either a LPN sample of a uniform sample

We define $\mathcal{B}$ as follows:

$\mathcal{B}(M^T, v)$:

1. $(y, y') \leftarrow \mathcal{A}(h_M)$
2. compute $v \cdot (\text{expand}(y) - \text{expand}(y'))$ (here the $\cdot$ denotes inner product)

Case 1: if v is uniform then

$$\Pr[v \cdot (\text{expand}(y) - \text{expand}(y')) = 0] = \frac{1}{2}$$

Case 2: if v is a LPN sample then v has the form $sM^T + e$ then

$$\begin{aligned} v \cdot (\text{expand}(y) - \text{expand}(y')) &= \\ sM^T + e \cdot (\text{expand}(y) - \text{expand}(y')) &= \\ 0 + e \cdot (\text{expand}(y) - \text{expand}(y')) & \end{aligned}$$

Then roughly speaking (details were skipped in lecture) due to sparseness we have

$$\Pr[e \cdot (\text{expand}(y) - \text{expand}(y')) = 0] \gg \frac{1}{2}$$

If we run $\mathcal{B}$ some n times on inputs $v_1, \dots, v_n$ then the expected value is

$$\theta := \mathbb{E}_i[v_i \cdot (\text{expand}(y) - \text{expand}(y'))]$$

So if $\theta \geq \frac{n}{2} - \frac{1}{\text{poly}(n)}$ then output “uniform” otherwise output “LPN”

□

12.1. cryptographic group actions

12.1.1. Definition: cryptographic group action

For a group G and a set X , a mapping $\star : G \times X \rightarrow X$ is said to be a group action if $\forall g, h \in G, x \in X$ the following holds:

$$g \star (h \star x) = (gh) \star x$$

(we notably skip the identity axiom where $e \star x = x$ for all $x \in X$ and identity $e \in G$)

12.1.2. Example: group action

The mapping $\star : \mathbb{Z}_q \times G \rightarrow G$ defined by $x \star g = g^x$

Where $(\mathbb{Z}_q, +)$ is the group

this looks a lot like discrete log (this is insecure against quantum adversaries)

12.1.3. Note

quantum resistant cryptographic constructions/assumptions often rely on X having little algebraic structure

12.1.4. Definition: regular group actions

a group action $\star : G \times X \rightarrow X$ is regular if the mapping $f_x : g \rightarrow g \star x$ defines a bijection between G and X

12.1.5. Definition: ‘effective group actions’ (EGA)

the following algorithms should be efficient:

membership testing in G

equality testing in G

sampling from G

group operation and inversion in G

membership testing in X

for all $x \in X$ x is uniquely representable

exists “origin” $x_0 \in X$

groups action operation

We define some hardness assumptions for effective group action (EGA)

12.1.6. Definition: One-way EGA

Given a pair $(x, g \star x)$ where $x \xleftarrow{\$} X$ and $g \xleftarrow{\$} G$ there is no PPT adversary $\mathcal{A}$ that can recover g

$\mathcal{A}$ is implicitly given f_x since we assume that f_x is efficiently computable/implementable

12.1.7. Definition: Weak Unpredictable EGA

Given poly many tuples of the form $(x_i, g \star x_i)$ where $g \xleftarrow{\$} G$ and each $x_i \xleftarrow{\$} X$ there is no PPT adversary that can compute $g \star x^*$ for a given challenge $x^* \leftarrow X$

This is implied from One-way EGA

12.1.8. Definition: Weak Pseudorandom EGA

there is no PPT adversary that can distinguish tuples of the form $(x_i, g \star x_i)$ from (x_i, u_i) where $g \xleftarrow{\$} G$ and each $x_i, u_i \xleftarrow{\$} X$

12.1.9. Definition: Pseudorandom functions from EGA

Let the key k be $(g_0, g_1, \dots, g_l) \xleftarrow{\$} G$

and let the input x (a bitstring) be of the form $(s_1, \dots, s_l)$

let x_0 be the “origin” of X

then we define the pseudorandom function

$$F(k, x) = \left(g_0 \prod_{i=1}^l g_i^{s_i} \right) \star x_0 = g_0 \star \left(g_1^{s_1} \star \left(g_2^{s_2} \star \dots \left(g_l^{s_l} \star x_0 \right) \right) \right)$$

where the second equality holds by the condition for a group action (12.1.1) and the implicit assumption that G is abelian, and denote $g_i^0 = e$ where e is the identity of G

12.1.10. Proposition: proof of security of 12.1.9 in the simple case

the function $F(k, x)$ as defined in 12.1.9 is secure if adversary $\mathcal{A}$ makes the all ones query $x = 11\dots 11$ i.e. $F(k, 11\dots 11) \approx_c u$ where $u \xleftarrow{\$} X$

Proof:

Follows by a natural hybrid argument and 12.1.8 i.e.

$$\begin{aligned} g_0 \star (g_1 \star (g_2 \star \dots (g_l \star x_0))) &\approx_c \\ g_0 \star (g_1 \star (g_2 \star \dots (u_l))) &\approx_c \\ &\vdots \approx_c \\ g_0 \star u_0 \end{aligned}$$

where $g_0 \star u_0$ is uniform on X

□

12.1.11. Definition: Public Key Encryption from EGA

We define a public key encryption scheme as follows:

Let $s \rightarrow G$ and $x \xleftarrow{\$} X$

The public key is $(x, \bar{x})$ where $\bar{x} = s \star x$

The secret key is s

Encryption on a bit $b \in \{0, 1\}$ is split in two cases:

If $b = 0$ output $(r \star x, r \star \bar{x})$

If $b = 1$ output $(r \star x, u)$ where $r \xleftarrow{\$} G, u \xleftarrow{\$} X$

Decryption can be done by checking one of the following two conditions on the parsed cipher text (c_1, c_2) :

$s^{-1} \star c_2 \stackrel{?}{=} c_1$ or $s \star c_1 \stackrel{?}{=} c_2$

If either of these conditions holds, then output 0 otherwise output 1

This is because in the case that $x = 0$, $s^{-1} \star c_2 = s^{-1} \star r \star s \star x = (s^{-1} s) \star r \star x = r \star x = c_1$

additionally, $s \star c_1 = s \star r \star x = r \star s \star x = r \star \bar{x} = c_2$

(G is once again implicitly abelian)

Then correctness follows

12.1.12. Proposition

security of 12.1.11

Proof:

If c is a ciphertext output by the encryption algorithm in 12.1.11, then $(pk, c) \approx_c (pk, \text{uniform})$ i.e.

$$\{(x, s \star x, r \star x, rs \star x)\} \approx_c \{(x, s \star x, r \star x, u)\}$$

where $u \xleftarrow{\$} X$

Since $r, x, s \star x$ are known parameters then this proof simplifies to showing that $\{(rs \star x)\} \approx_c \{u\}$

It follows by 12.1.8

□

13. Lecture 13: Quantum Query Security

13.0.1. Notation: Oracle to access to function f

$\mathcal{A}^f$ for some function f means that adversary $\mathcal{A}$ has can query the oracle f which will respond with $f(x)$ if x is the query.

These queries can be made multiple times (if the same query x is made more than once, $f(x)$ will not change)

13.0.2. Definition: Pseudorandom Function

A function $\text{PRF} : \{0, 1\}^\lambda \times \{0, 1\}^n \rightarrow \{0, 1\}^m$ is said to be pseudorandom if

$$\mathcal{A}^{\text{PRF}(k, \cdot)} \approx_c \mathcal{A}^{g(\cdot)}$$

where $k \xleftarrow{\$} \{0, 1\}^\lambda$ and g is a random function $g : \{0, 1\}^n \rightarrow \{0, 1\}^m$

13.0.3. Definition: Random Oracle Model

We give adversary $\mathcal{A}$ black box access to hash function H (denoted $\mathcal{A}^H$) where H is modeled as a random function.

This idealized settings often helps in theoretical crypto

In the real world, adversaries typically have access to the “source code” of hash function H

13.0.4. Note

In the post quantum setting, adversaries can query on a quantum superposition of inputs, thereby giving the adversary a superposition of the values of the function at many inputs at once

This was referred to in lecture as “superposition” “attacks”

Effectively, adversaries no longer necessarily make polynomially many queries to a function but can feasibly query exponentially many times

13.0.5. Definition: Quantum Oracle Access (standard oracle)

We let $f : \{0, 1\}^n \rightarrow \{0, 1\}$ be some classical function and U_f be the unitary operator representing which implements f

In the post quantum setting, $\mathcal{A}$ can make the query

$$\sum_{x \in \{0,1\}^n} \alpha_x |x\rangle |0\rangle$$

and receive back

$$\sum_{x \in \{0,1\}^n} \alpha_x |x\rangle |f(x)\rangle$$

More Generally, $\mathcal{A}$ queries a unitary U_f where $U_f |x\rangle |y\rangle \mapsto |x\rangle |y \oplus f(x)\rangle$

The XOR is needed in order to ensure reversibility of U_f

It is a commonly cited fact that if f is efficiently computable/implementable then U_f is also efficiently computable/implementable

13.0.6. Definition: Phase Oracle

The phase oracle is defined as

$$U_f^\pm : |x\rangle \mapsto (-1)^{f(x)} |x\rangle$$

For example, if we query the standard oracle on $\sum_x |x\rangle |-\rangle$ then: (To save space, we omit the sum in the following derivation to save some typing)

$$U_f \left(|x\rangle \frac{1}{\sqrt{2}} (|0\rangle - |1\rangle) \right) = \frac{1}{\sqrt{2}} (|x\rangle |0 \oplus f(x)\rangle - |x\rangle |1 \oplus f(x)\rangle)$$

$$\text{If } f(x) = 0 \Rightarrow \frac{1}{\sqrt{2}} (|x\rangle |0\rangle - |x\rangle |1\rangle) = |x\rangle |-\rangle$$

$$\text{Similarly } f(x) = 1 \Rightarrow \frac{1}{\sqrt{2}} (|x\rangle |1\rangle - |x\rangle |0\rangle) = -|x\rangle |-\rangle$$

$$\text{Overall, we have } U_f |x\rangle |-\rangle = (-1)^{f(x)} |x\rangle |-\rangle$$

13.0.7. Proposition: Extracting s from $\langle x, s \rangle$ (Bernstein-Vazirani)

Question: can we learn certain properties about some function f more efficiently in the quantum query setting?

Let $f_s : \{0, 1\}^n \rightarrow \{0, 1\}$ be defined as $f_s(x) = \langle x, s \rangle = x \cdot s$ (bitwise dot product)

Proof:

In classical setting, if s is length n then we will need n XOR operations to reveal each bit of s individually

However, one invocation of Bernstein-Vazirani algorithm is sufficient to reveal s

The Bernstein-Vazirani algorithm takes as input $|0\rangle^{\otimes n}$ (an n qubit state) and then applies three sequential operations: n qubit hadamard, $U_f^\pm$, and another n qubit hadamard:

$$\begin{aligned} |0\rangle^{\otimes n} &\xrightarrow{H^{\otimes n}} \sum_{x \in \{0,1\}^n} \frac{1}{\sqrt{2^n}} |x\rangle \xrightarrow{U_f^\pm} \sum_{x \in \{0,1\}^n} \frac{1}{\sqrt{2^n}} (-1)^{f_s(x)} |x\rangle \xrightarrow{H^{\otimes n}} \\ &\sum_{x \in \{0,1\}^n} \frac{1}{\sqrt{2^n}} (-1)^{x \cdot s} \left(\sum_{y \in \{0,1\}^n} \frac{1}{\sqrt{2^n}} (-1)^{x \cdot y} |y\rangle \right) = \\ &\frac{1}{2^n} \sum_{y \in \{0,1\}^n} \left(\sum_{x \in \{0,1\}^n} (-1)^{x \cdot s} (-1)^{x \cdot y} |y\rangle \right) = \\ &\frac{1}{2^n} \sum_{y \in \{0,1\}^n} \left(\sum_{x \in \{0,1\}^n} (-1)^{(s \oplus y) \cdot x} |y\rangle \right) \end{aligned}$$

where the second equality comes from the fact that we are working addition mod 2

In the case that $y = s$ we obtain

$$\frac{1}{2^n} \sum_{y \in \{0,1\}^n} \left(\sum_{x \in \{0,1\}^n} 1 |y\rangle \right) = \frac{1}{2^n} \sum_{y \in \{0,1\}^n} (2^n) |y\rangle = 2^n |y\rangle$$

, revealing $|s\rangle$ after measurement

In the case that $y \neq s$ we obtain

$$\frac{1}{2^n} \sum_{y \in \{0,1\}^n} \left(\sum_{x \in \{0,1\}^n} (-1)^{\alpha \cdot x} |y\rangle \right) = 0$$

□

13.0.8. Note

A nice example running the Bernstein-Vazirani algorithm can be found in section 3.4.1 in https://physlab.org/wp-content/uploads/2023/05/BernsteinVazirani_23100071_Fin.pdf

13.1. Quantum Query Secure Pseudorandom functions

13.1.1. Notation: quantum query access

We let $\mathcal{A}^f$ denote classical query access to f

Let $\mathcal{A}^{[f]}$ denote quantum query access to f

13.1.2. Concept: Quantum Query Security

We are interested if PRFs such that

$$\{\mathcal{A}^{\text{PRF}(k, \cdot)}\} \approx_c \{\mathcal{A}^g\}$$

where $k \xleftarrow{\$} \{0, 1\}^\lambda$ and g is a random function $g : \{0, 1\}^n \rightarrow \{0, 1\}^m$

$\mathcal{A}$ is a quantum polynomial time algorithm (its number of gates is polynomial in the length of the input)

Such PRFs we can call quantum PRF or QPRF

[Zhandry' 13] showed that $\exists$ PRF construction which satisfies classical query security but doesn't satisfy quantum query security

The LWR based PRF has been shown to satisfy quantum query security (recall that $\text{PRF}(k, x) = \left[A \prod_i S_i x_i \right]_p$ from earlier (4.0.10))

We will show that PRF based on one way function does satisfy quantum query security

13.1.3. Definition: GGM construction of PRF

GGM stands for Goldreich-Goldwasser-Micali

The following is a PRF:

$$\text{PRF}(k, x) = G^{x_l}(G^{x_{l-1}}(\dots(G^{x_1}(k))\dots))$$

The construction is tree based and utilizes a length pseudorandom generator $G : \{0, 1\}^\lambda \rightarrow \{0, 1\}^{2\lambda}$ with a key $k \in \{0, 1\}^\lambda$ and input $x = x_1, \dots, x_l \in \{0, 1\}^l$. We let G^0 and G^1 denote the first and second halves of the output of G respectively

The proof of security follows from the security of G which is a pseudorandom generator (i.e. it is indistinguishable from uniform)

In each hybrid $H_i, i \in 1, \dots, l$ we give $\mathcal{A}$ access to a function F_i which are as follows:

$$\begin{aligned} F_1 &= \text{PRF}(k, x) = G^{x_l}(G^{x_{l-1}}(\dots(G^{x_1}(k))\dots)) \\ F_2 &= G^{x_l}(G^{x_{l-1}}(\dots(P_1(x_1))\dots)) \\ &\vdots \\ F_i &= G^{x_l}(G^{x_{l-1}}(P_i(x_1, \dots, x_i))) \\ &\vdots \\ F_l &= P_l(x_1, \dots, x_l) \end{aligned}$$

where $P_i : \{0, 1\}^i \rightarrow \{0, 1\}^\lambda$ is a random function

Then F_l is equivalent to a random function

■

13.1.4. Definition: Indistinguishability of distributions

Two distributions D_1, D_2 over a set $\mathcal{Y}$ are said to be computationally (resp. statistically) indistinguishable if no efficient (resp. computationally unbounded) quantum algorithm A can distinguish a sample of D_1 from a sample of D_2

In other words, for all A there is a negligible function ε such that

$$\left| \Pr_{y \leftarrow D_1} [A(y) = 1] - \Pr_{y \leftarrow D_2} [A(y) = 1] \right| \leq \varepsilon$$

(from definition 2.4 from How to Construct Quantum Random Functions by Mark Zhandry)

13.1.5. Definition: Oracle Indistinguishability

Two distributions D_1, D_2 over a set $\mathcal{Y}$ are computationally (resp. statistically) oracle-indistinguishable if, for all sets $\mathcal{X}$ no efficient (resp. computationally unbounded) quantum algorithm B can distinguish $D_1^{\mathcal{X}}$ from $D_2^{\mathcal{X}}$ using a polynomial number of quantum queries. that is, for all B and $\mathcal{X}$ there is a negligible function ε such that

$$\left| \Pr_{O \leftarrow D_1^{\mathcal{X}}} [B^{(O)}() = 1] - \Pr_{O \leftarrow D_2^{\mathcal{X}}} [B^{(O)}() = 1] \right| \leq \varepsilon$$

(definition 2.5)

13.1.6. Theorem: distinguishable iff oracle-distinguishable

Let D_1 and D_2 be efficiently sampleable distributions over a set $\mathcal{Y}$. Then D_1 and D_2 are indistinguishable if and only if they are also oracle-indistinguishable

Proof: very complicated

□

13.1.7. Definition: Small Range Distribution

Fix sets $\mathcal{X}$ and $\mathcal{Y}$ and a distribution D on $\mathcal{Y}$. Fix an integer r . Let $y = (y_1, \dots, y_r)$ be a list of r samples from D and let P be a random function from $\mathcal{X}$ to $[r]$. The distributions on y and P induce a distribution on functions $H : \mathcal{X} \rightarrow \mathcal{Y}$ defined by $H(x) = y_{P(x)}$. This distribution is called a small-range distribution with r samples of D

13.1.8. Lemma: Small Range

There is a universal constant C_0 such that, for any sets $\mathcal{X}$ and $\mathcal{Y}$, distribution D on $\mathcal{Y}$, any integer l , and any quantum algorithm A making q queries to an oracle $H : \mathcal{X} \rightarrow \mathcal{Y}$, the following two cases are indistinguishable, except with probability less than $\frac{C_0 q^3}{l}$:

1. $H(x) = y_x$ where y is a list of samples of D of size $|\mathcal{X}|$.
2. H is drawn from the small-range distribution with l samples of D

13.1.9. Note

In the lecture we denoted this result as $\mathcal{A}^{|D\rangle}_{s,\varepsilon} \approx \mathcal{A}^{|\text{SR}_r(D)\rangle}$ where $\varepsilon = O\left(\frac{q^3}{r}\right)$ where $r = l$ as above with $r > q^3$

Proof: See the reference reading

□

13.1.10. Theorem

If G is a standard-secure PRG, then PRF from 13.1.3 is a QPRF as described in 13.1.2

Proof:

(sketch only)

Let $F_1, \dots, F_l$ be the random functions from 13.1.3

We would like to show that $\mathcal{A}^{|F_1\rangle} \approx_c \mathcal{A}^{|F_2\rangle} \approx_c \dots \approx_c \mathcal{A}^{|F_l\rangle}$

Where $\mathcal{A}$ is an adversary which has quantum query access to $F_1, \dots, F_l$

Using the small range lemma (13.1.8) we can see that

$$\mathcal{A}^{|D_i\rangle} \approx_c \mathcal{A}^{|\text{SR}_r(D_i)\rangle}$$

where D_i is the distribution of the outputs of F_i so that $\mathcal{A}^{|D_i\rangle}$ is equivalent to $\mathcal{A}^{|F_i\rangle}$ and r is polynomial

If we can show that $\mathcal{A}^{|\text{SR}_r(D_i)\rangle} \approx_c \mathcal{A}^{|\text{SR}_r(D_j)\rangle}$ for all $i, j \in 1, \dots, l$

Then $\mathcal{A}^{|D_1\rangle} \approx_c \mathcal{A}^{|D_l\rangle}$

□

14. Lecture 14 Quantum Query Security continued, signatures

14.0.1. Definition: Digital signature security (classical)

Here we repeat the definition of classical digital signatures

for a signing key sk and verification key vk the adversary $\mathcal{A}$ makes polynomially queries $m_1, \dots, m_q$ to the challenger

The challenger sends to $\mathcal{A}$ both the vk and $\sigma_1, \dots, \sigma_q$ where $\sigma_i \rightarrow \text{sign}(sk, m_i)$

We say that if the probability that PPT $\mathcal{A}$ outputs (m^*, σ^*) such that

$$\text{verify}(vk, m^*, \sigma^*) = \text{VALID} \text{ and } m^* \notin \{m_1, \dots, m_q\}$$

is negligible, then the signature scheme (Gen, sign, verify) is secure (sometimes referred to as chosen message secure)

14.0.2. Definition: Quantum Digital Signature Security

The definition is very similar to 14.0.1

For $sk, vk \leftarrow \text{Gen}()$ the challenger gives a efficient quantum algorithm $\mathcal{A}$ vk

$\mathcal{A}$ is then makes some polynomially many q quantum message queries $\sum_{m,y} \alpha_{my} |m\rangle |y\rangle$ and receives back from the challenger $\sum_{m,y} \alpha_{my} |m\rangle |y \oplus \text{sign}(sk, m; r)\rangle$

Crucially, we note for each query $\mathcal{A}$ must sample the returned superposition of signatures

$\sum_{m,y} \alpha_{my} |m\rangle |y \oplus \text{sign}(sk, m; r)\rangle$ which means that $\mathcal{A}$ will obtain one message/signature pair (m_i, σ_i)

Then we require that the probability that $\mathcal{A}$ outputs $(m_1, \sigma_1, \dots, m_{q+1}, \sigma_{q+1})$ after q queries such that $\text{verify}(vk, m_i, \sigma_i) = \text{VALID}, i \in 1, \dots, q+1$ is negligible. Additionally, all $q+1$ message/signature pairs must be distinct

to summarize, in order to win $\mathcal{A}$ must produce $q+1$ unique valid signature pairs

14.0.3. Proposition: Equivalent methods for implementing a randomized sign

The following two methods for implementing a randomized sign algorithm are equivalent:

- 1) choose a single randomness value for each chosen message query, and sign every message in the superposition with that randomness.
- 2) choose fresh randomness for each message in the superposition

Proof:

To sign a message m , compute and output $\text{QPRF}(k, m)$ where k is fresh random key sample once per query and QPRF is a quantum pseudorandom function.

□

14.0.4. Concept: Seperation

In the context of quantum cryptogrphahy, “seperation” means secure under classical queries, but completely insecure once an adversary can make quantum queries

14.0.5. Definition: Seperation Signature Scheme

Fix positive integers N, N' and let the message space $\mathcal{M} = \{0, 1, \dots, N - 1\}$

Let $\mathcal{S} = (\text{Keygen}, \text{Sign}, \text{Verify})$ be a signature scheme that signs messages in $\mathcal{M}$ and let PRF be a pseudorandom function with domain $\mathcal{M}$

Define a new signature scheme $\mathcal{S}^\sim = (\text{Keygen}^\sim, \text{Sign}^\sim, \text{Verify}^\sim)$ as follows:

$\text{Keygen}^\sim(\lambda)$:

sample $k \xleftarrow{\$} \{0, 1\}^\lambda$, $p \rightarrow$ random prime in $\left[\frac{N'}{2}, N'\right)$ and let $(sk, vk) \leftarrow \text{Keygen}(\lambda)$

Output $vk^\sim = vk$ and $sk^\sim = (sk, k, p)$

$\text{Sign}^\sim((sk, k, p), m)$:

$s_1 \leftarrow \text{PRF}(k, m \bmod p)$

$s_2 \leftarrow \begin{cases} sk & \text{if } m=p \\ 0 & \text{if } m \neq p \end{cases}$

$\sigma \leftarrow \text{Sign}(sk, (m, s_1, s_2))$

output (σ, s_1, s_2)

$\text{Verify}^\sim(pk, m, (\sigma, s_1, s_2))$:

output $\text{Verify}(vk, (m, s_1, s_2), \sigma)$

14.0.6. Theorem: 14.0.5 is not secure according to 14.0.2

If $\mathcal{S}$ is secure according to definition 14.0.1, then $\mathcal{S}^\sim$ is secure against 14.0.1 but insecure against 14.0.2

Proof:

Both classical and quantum security were proven in [Boneh, Zandry 2013 - “Secure Signatures and Chosen Ciphertext Security in a Quantum Computing World”]

In class we only discussed the quantum query security.

[Boneh, Lipton 95] showed that there exist a period finding algorithm which does the following: given some function f_p such that $f_p(x) = f_p(x + p) \forall x$ and quantum oracle access to f_p , p is efficiently recoverable.

In 3.0.5. f_p is $\text{PRF}(k, m \bmod p)$

This implies if we can extract the s_1 for all m then p is recoverable. Then an we can query on $m = p$ (classically) and obtain sk , breaking the scheme

To obtain s_1 we devise the following “ s_1 oracle”:

Query the oracle(which runs verify) on $\sum \alpha_{m,y} |m\rangle |y\rangle$ which returns $\sum \alpha_{m,y} |m\rangle |y \oplus (s_1^m, s_2^m, \sigma^m)\rangle |s_1^m\rangle$

Then, choose some y' and query the oracle on $\sum \alpha_{m,y} |y \oplus (s_1^m, s_2^m, \sigma^m)\rangle |s_1^m \oplus y'\rangle$ to obtain

$$\sum \alpha_{m,y} |m\rangle |y\rangle |y' \oplus s_1^m\rangle$$

as desired

□

14.0.7. Note

For construction in 3.0.5 $\mathcal{A}$ can also try to guess some messages $m_1, \dots, m_q$

If any of the $m_i = p$ or $m_i \equiv m_j \pmod{p}, i \neq j$ for some guessed p then $\mathcal{A}$ also wins. This was shown in [Boneh, Zandry 2013] to only occur with negligible probability.

14.1. Signature scheme satisfying quantum query security

We first define a “chameleon” hash function:

14.1.1. Definition: Chameleon Hash Function

A chameleon hash function is a tuple of efficient algorithms $(G, H, \text{Inv}, \text{Sample})$

where $G(\lambda) \rightarrow (sk, pk)$

$H(pk, m, r)$ maps messages to some space $\mathcal{Y}$

$\text{Sample}(\lambda)$ samples r from some distribution such that for every pk and m , $H(pk, m, r)$ is uniformly distributed

$\text{Inv}(sk, h, m) \rightarrow r$ such that $H(pk, m, r) = h$ and r is distributed negligibly close to $\text{Sample}(\lambda)$ conditioned on $H(pk, m, r) = h$

The chameleon hash function is said to be collision resistant if no efficient quantum algorithm given only pk can find collisions in $H(pk, \cdot, \cdot)$

14.1.2. Note

One construction of chameleon hash function relies on the hardness of SIS (Cash et al.)

14.1.3. Note

$D_1 \approx_s D_2$ follows directly from definition: where $D_1 = (h, m, r)$ such that $\text{Inv}(sk, h, m) \rightarrow r$ and $D_2 = (h, m, r)$ such that $r \leftarrow \text{sample}(), H(pk, m, r) = h$

14.1.4. Definition: sig scheme (quantum query secure)

Let $\mathcal{S} := (\text{KeyGen}, \text{Sign}, \text{Verify})$ be secure against classical adversaries

We want to design $\mathcal{S}^\sim := (\text{KeyGen}^\sim, \text{Sign}^\sim, \text{Verify}^\sim)$ which is quantum query secure

$\text{KeyGen}^\sim(\lambda) :$

Run $\text{KeyGen}() \rightarrow (sk, vk)$ and $G(\lambda) \rightarrow (sk_H, pk_H)$ where the second tuple is the key pair for the chameleon hash function $(G, H, \text{Inv}, \text{Sample})$

Output $sk = m, sk^\sim(sk, pk_H)$ and $vk^\sim = (vk, pk_H)$

$\text{Sign}^\sim(m, sk^\sim) :$

$r \leftarrow \text{Sample}(), h = H(pk_H, m, r), \sigma \leftarrow \text{Sign}(sk, h)$

output $(r, \sigma) = \sigma^\sim$

$\text{Verify}^\sim(vk^\sim, m, \sigma^\sim) :$

parse $\sigma^\sim = (r, \sigma)$

compute $H(pk_H, m, r) = h$

output the result of $\text{Verify}(vk, h, \sigma)$

Note: this is an example of “hash then sign”

14.1.5. Theorem: security of 14.1.4

14.1.4 is secure according to 14.0.2

Proof:

(sketch only)

First we should note that in the original paper [Zhandry, Boneh 13] that $\mathcal{S}$ is secure against classical adversaries in the sense that it is secure against attackers where the adversary is restricted to query on uniformly random messages (in this case hashes)

The goal of the proof is that under the assumption that $\mathcal{S}^\sim$ is not quantum query secure against $\mathcal{A}$, show that the security of $\mathcal{S}$ does not hold.

First, we consider an adversary $\mathcal{A}$ which performs the classical chosen message attack (14.0.1) on $m_1, \dots, m_q$. The challenger responds by computing $H(pk, m_i, r_i) \rightarrow h_i$ uniformly using $\text{Sample}()$ and optionally run $\text{Inv}(sk_H, h_i, m_i) \rightarrow r_i^m$, then $\text{Sign}(sk, h_i) \rightarrow \sigma_i$ and outputs (r_i, σ_i) (or (r_i^m, σ_i)) for $i \in 1, \dots, q$. Therefore for any message the challenger is able to respond.

If the adversary $\mathcal{A}$ performs a quantum query attack via quantum queries $q_i = \sum \alpha_{m,y} |m\rangle |y\rangle$ for $i = 1, \dots, q$, the challenger will need to sign an exponential number of messages. However, using the small range lemma (13.1.8) the number of l hashes needed to sign is reduced to a large polynomial (see 13.1.8).

By assumption $\mathcal{A}$ is able to $q + 1$ valid message/signature pairs (unique) $(m_1^*, \sigma_1^\sim, \dots, m_{q+1}^*, \sigma_{q+1}^\sim)$ each corresponding to random hashes $h_1^*, \dots, h_{q+1}^*$

Let $\mathcal{B}$ be an algorithm which breaks the security of $\mathcal{S}$. Let ch_c be the classical challenger and ch_q be the quantum challenger. $\mathcal{B}$ will make $h_1, \dots, h_q$ queries to ch_c and receive back $\sigma_1, \dots, \sigma_q$

Since $l > q + 1$ (see 13.1.8) then we can consider the 2 cases:

- 1: $\exists i$ such that h_i was signed in the interaction between $\mathcal{A}$ and ch_q that is not $h_1, \dots, h_q$. Then we have a forgery for message h_i and the security of classical scheme $\mathcal{S}$ is broken
- 2: $\nexists i$ such that h_i was signed in the interaction between $\mathcal{A}$ and ch_q that is not $h_1, \dots, h_q$. We can pick a one of the q responses to the queries that $\mathcal{A}$ made and perform a measurement and succeed with $\frac{1}{q}$ probability.

[Zhandry, Boneh 13] argues that two of the message/signature output by $\mathcal{A} \rightarrow (m_1^*, \sigma_1^\sim, \dots, m_{q+1}^*, \sigma_{q+1}^\sim)$ are in some sense “the same” in the sense that they correspond to the same query.

□